PROYECTO FINAL – DESARROLLO Y ARQUITECTURA BACKEND

MSc. Feisar Moreno

El objetivo principal del proyecto final de la asignatura es construir una solución *backend* robusta, escalable y mantenible sobre el lenguaje PHP y el *framework* Laravel (o una tecnología similar previamente autorizada por el profesor).

En cuanto a la temática específica, los estudiantes tienen la libertad de seleccionar la temática y el dominio de negocio de su proyecto. La propuesta debe poseer la riqueza operativa suficiente para justificar una arquitectura *backend* en capas y evitar la implementación de sistemas CRUD triviales.

Requisitos Arquitectónicos y Técnicos Mínimos

La arquitectura del sistema desarrollado debe integrar obligatoriamente los componentes técnicos abordados en la materia, cumpliendo con los siguientes mínimos indispensables:

1. Modelo de Datos y Persistencia con Eloquent (u otro ORM similar)

- **Estructura de Base de Datos:** Diseñar e implementar como mínimo 8 modelos y sus correspondientes tablas en el sistema de gestión de base de datos relacional o no relacional seleccionado (MySQL, PostgreSQL, MongoDB, etc.).
- **Mapeo Objeto-Relacional:** La interacción con la base de datos y la gestión de la persistencia deben realizarse utilizando el ORM nativo de Laravel (Eloquent), definiendo modelos con sus respectivas propiedades, asignación masiva (*$fillable* o *$guarded*) y tipos de relaciones (uno a uno, uno a muchos y muchos a muchos).
- **Poblado y Estructuración:** Incluir archivos de migraciones, *factories* para la generación de datos de prueba y un seeder principal (*DatabaseSeeder.php*) que garantice la inicialización limpia del entorno de desarrollo.

2. Exposición de API REST

- **Endpoints Expuestos:** Implementar al menos 4 endpoints funcionales orientados al consumo mediante una arquitectura API REST protegida o pública, atendiendo a las mejores prácticas de diseño de URIs y códigos de estado HTTP (200 OK, 201 Created,

400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Internal Server Error).

- **Controladores API:** Desacoplar la lógica de enrutamiento y procesamiento mediante controladores que gestionen adecuadamente la transformación y retorno de respuestas estructuradas en formato JSON.

3. Consumo de Servicios Externos (APIs de Terceros)

- **Integración Doble:** El sistema debe conectarse y consumir como mínimo 2 servicios o APIs externas para complementar la lógica de negocio (por ejemplo, pasarelas de pago como Stripe, servicios de notificación por correo electrónico como SendGrid, APIs de geolocalización o servicios meteorológicos).
- **Capa de Servicios (Service Layer):** La comunicación con servicios externos debe encapsularse en clases de servicio dedicadas dentro de la arquitectura *backend*. No se debe incluir lógica de integración HTTP directa dentro de los controladores o modelos.
- **Seguridad y Webhooks:** Gestionar las credenciales secretas mediante el archivo de configuración de entorno *.env* y *config/services.php*. En caso de recibir notificaciones asíncronas de terceros (*webhooks*), se debe configurar la exención correspondiente del *middleware* de validación CSRF.

4. Autenticación, Autorización y Gestión de Perfiles

- **Gestión de Identidad:** Implementar procesos de registro, inicio de sesión y autenticación de usuarios mediante *tokens* criptográficos (utilizando Laravel Sanctum o similar).
- **Perfiles y Roles de Usuario:** La aplicación debe manejar al menos 3 perfiles o roles de usuario diferenciados (por ejemplo, Administrador, Proveedor/Gestor y Cliente/Usuario final).
- **Políticas de Autorización:** Controlar el acceso a los recursos mediante herramientas de autorización de Laravel.

5. Esquema de Pruebas Automatizadas y Mocks

- **Suite de Pruebas:** Crear un conjunto de pruebas automatizadas que cubra el 100% de los *endpoints* REST implementados, utilizando el entorno de pruebas de Laravel (PHPUnit) o similar.
- **Aislamiento de Entorno:** Configurar el entorno para ejecutar las pruebas utilizando una base de datos en memoria (SQLite o similar) y ejecutando las migraciones automáticamente en cada prueba.
- **Uso de Mocks:** Simular el comportamiento de los servicios externos consumidos (APIs de terceros) mediante clases o fachadas de simulación (Mocks / HTTP Facade Fakes) para garantizar ejecuciones de prueba independientes de conexiones a internet.

6. Formularios, Validaciones y Procesamiento de Solicitudes

- **Form Requests:** Validar estrictamente la estructura y tipos de datos recibidos en las peticiones mediante clases de validación dedicadas (*FormRequest*), definiendo mensajes de error personalizados y control de archivos adjuntos cuando aplique.

7. Capa de Presentación (Frontend)

- **Plataforma a Elección:** Se debe construir un *frontend* funcional que consuma los servicios *backend* en la plataforma o plataforma de su elección:
    - **Aplicación Web:** Desarrollada en HTML/JS, Vue.js, React, Angular, etc.
    - **Aplicación Móvil:** Desarrollada en Android nativo (Kotlin), iOS nativo (Swift) o multiplataforma (Flutter / React Native).

Cronograma de Entregas e Hitos de Desarrollo

El desarrollo del proyecto final se encuentra estructurado en tres entregas consecutivas.

- **Primera Entrega (20%) — Propuesta y Arquitectura (3 de noviembre de 2026, 4:00 pm)**

  Documento formal (formato PDF) que especifique:

- **Descripción del Dominio de Negocio:** Planteamiento claro del problema que se abordará y justificación del sistema.
- **Especificación de Actores y Roles:** Definición explícita de los 3 perfiles de usuario y sus permisos en el sistema.
- **Diseño del Esquema de Datos:** Diagrama relacional y definición del diccionario de datos de las tablas/modelos.
- **Plan de Integración de APIs Externas:** Identificación y descripción de las APIs externas que se conectarán.
- **Definición de Endpoints REST:** Listado de los *endpoints* previstos con su método HTTP, URI y resumen de funcionalidad.

- **Segunda Entrega (30%) — Avance de Núcleo Backend (10 de noviembre de 2026, 4:00 pm)**

  Código fuente funcional del proyecto en el repositorio oficial:

  - **Modelos y Migraciones:** Implementación terminada de las migraciones, modelos Eloquent y sus relaciones.
  - **Datos:** Factories y Seeders ejecutables mediante `migrate:fresh --seed`.
  - **Sistema de Autenticación y Autorización:** Configuración funcional de Laravel Sanctum para emisión de *tokens* y definición de Roles con *Gates* y *Policies*.
  - **Controladores y Enrutamiento:** Implementación básica de los controladores y la estructura inicial de las rutas API.

- **Tercera Entrega (30%) — Entrega Final (17 de noviembre de 2026, 1:00 pm)**

  Proyecto completo y funcional con los siguientes componentes entregados y probados:

  - **Integración con APIs Externas:** Capa de servicio integrada y operando con los servicios externos seleccionados.
  - ***Endpoints* REST Completos:** 4 *endpoints* REST completamente funcionales, validados mediante *FormRequest* y retornando respuestas JSON estandarizadas.

- **Suite de Pruebas Automatizadas:** Pruebas de integración de todos los endpoints ejecutables en base de datos *SQLite* en memoria con *Mocks* para APIs externas.
- **Frontend Integrado:** Aplicación Web o Móvil conectada al *backend* realizando el flujo de interacción completo.

- **Sustentación en Clase (20%) (17 de noviembre de 2026, de 1:00 pm a 4:00 pm)** Demostración en vivo del funcionamiento de la aplicación y defensa técnica del código desarrollado.

Instrucciones de Entrega y Políticas de Código

La entrega del documento inicial de especificación debe realizarse a través de la plataforma TEMA en formato PDF.

Todas las entregas de código deben registrarse mediante un repositorio público en GitHub, enviado a través de la plataforma institucional TEMA antes de la hora límite establecida para cada fecha.

El repositorio debe incluir un archivo README.md detallado con las instrucciones de instalación, configuración del archivo .env.example, ejecución de migraciones/seeders y comandos de ejecución de la suite de pruebas.